

Übersetzung und Analyse von Programmen

The background is a stylized, low-poly illustration. It features a multi-story building with blue and orange facades and large windows. In the foreground, there are two green trees with yellow foliage, a large white flower with a yellow center, and a bright yellow sun in the upper right corner. The sky is composed of various shades of blue and green geometric shapes.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl

Informatik, Universität Trier

Byte-Code Verification: Stack Analyse

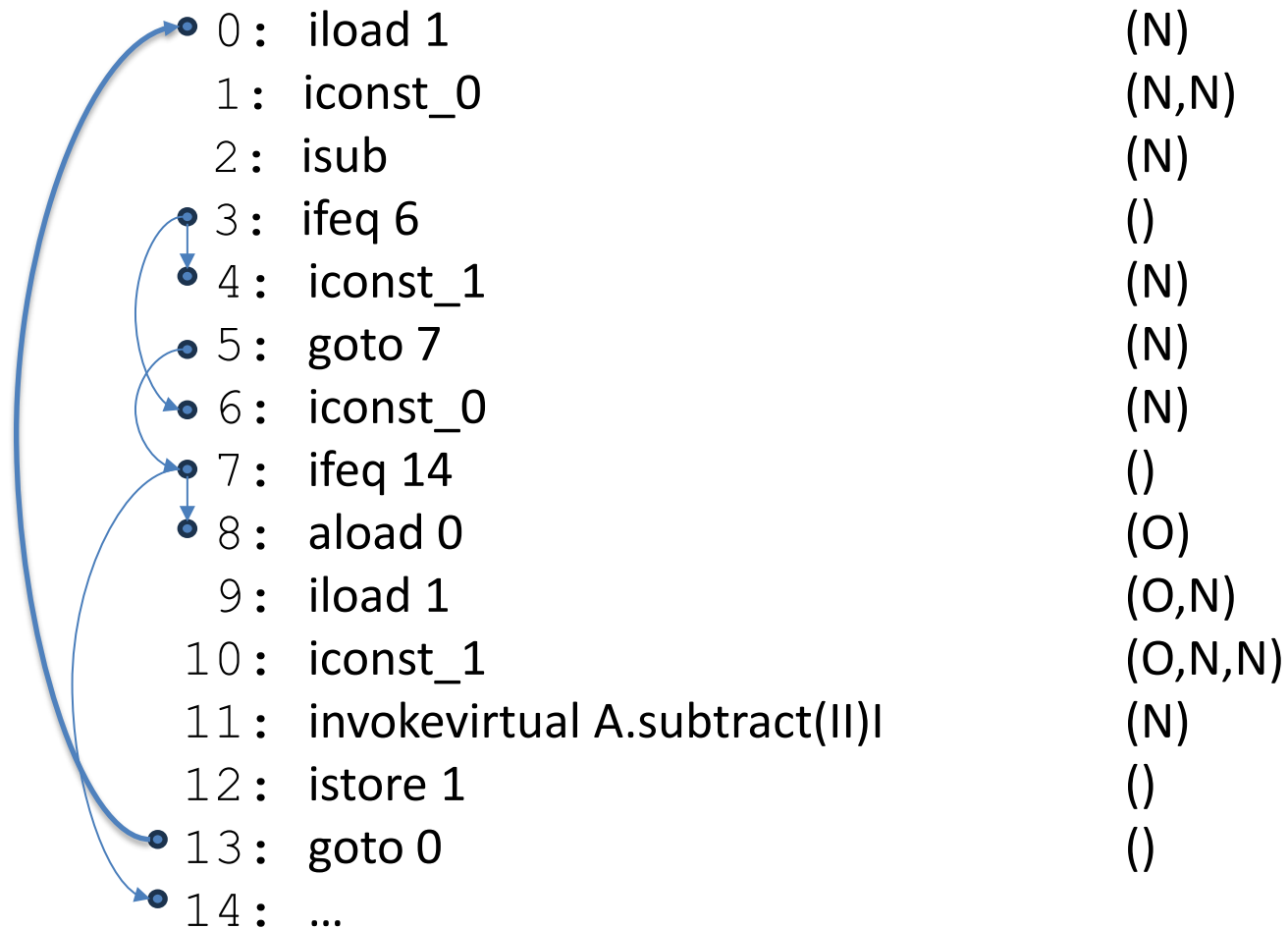
Integer Arithmetics

Instruction	Meaning	Before	After
iadd	$STACK[OPTOP - 1] :=$ $STACK[OPTOP - 1] + STACK[OPTOP];$ $OPTOP := OPTOP - 1;$ $PC := PC + 1$	(N,N)	(N)
analogous: isub , imul , idiv , iand , ior		Verarbeitet die beiden oberen Zahlen ausgehend von OPTOP und schreibt das Ergebnis als obersten Wert auf den Stack	

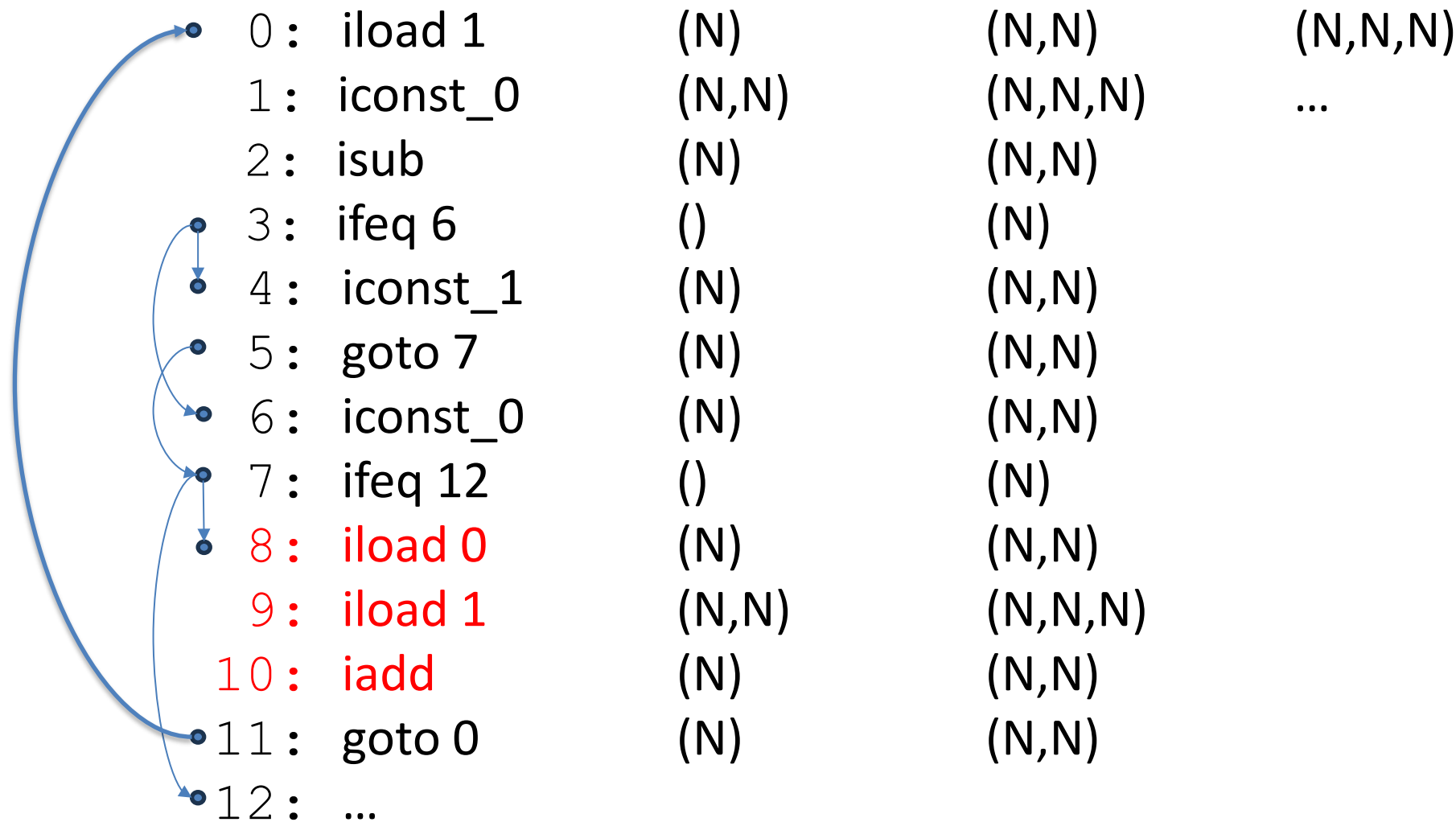
```
class A {  
    int subtract(int a, int b)  
        { return a-b+b-b; }  
}
```

```
0 : iload 0  
1 : iload 1  
2 : iload 1  
3 : iload 1  
4 : isub  
5 : iadd  
6 : isub  
7 : ireturn
```

while (x!=0) { x=this.subtract(x,1); }



Ungültiges Byte-Code Programm





Timothy Cramer
Richard Friedman
Terrence Miller
David Seberger
Robert Wilson
Mario Wolczko
Sun Microsystems

The Java programming language prioritizes portability, secure execution of applications. Early Java implementations relied on interpretation, leading to poor performance compared to compiled programs. Compiling Java programs to the native machine instructions provides much higher performance. Because traditional compilation would defeat Java's portability and security, another approach is necessary. This article describes some of the important issues related to just-in-time, or JIT, compilation techniques for Java. We focus on the JIT compiler developed by Sun for use with the JDK (Java Development Kit) target machine running on SPARC and Intel processors. (Access the Web at www.sun.com/worksheets/javajit for these compilers and additional information.) We also discuss performance improvements and limitations of JIT compilers. Future Java implementations may provide even better performance, and we outline some specific techniques that they may use.

form of variable length, with the shortest instructions occupying 1 byte and most instructions being 1 to 3 bytes long. This form of encoding is known as bytecode. Previous systems such as the UCSD Pascal System and most Smalltalk implementations have used similar bytecodes.

A Java source-to-bytecode compiler, such as the `javac` program of the JDK, compiles the classes that constitute a Java program. The compiler translates methods in each source class into bytecode instructions and places all the bytecodes for a class together in a class file.

To run a Java program, the JVM loads the class file containing the program's entry point, and execution begins. The program may reference other class files, which are loaded in turn (possibly across a network). Hence, the final association of class files to execution proceeds. To maintain the integrity of the Java execution model, the JVM checks that a variety of semantic constraints are satisfied both within a class file and between class files. For example, bytecode from one class cannot access a field defined in another class unless explicitly permitted by the access specification in the field definition. As another example, when an integer is pushed onto the stack, all bytescets that refer to that value must treat it as an integer, and not, say, as a float reference.

As part of a program's execution, the JVM must provide various services. It must manage memory, allowing programs to create objects and reclaiming objects once they are no longer required (a process known as garbage collection). Java also allows interoperation with machine code created from other source languages (such as C), that is encapsulated to appear to a Java program as Java methods. Therefore, the JVM must also mediate between Java methods and these

java execution model

Java execution model

The Java Virtual Machine (JVM) assures Java application portability and security. The JVM provides a well-defined runtime framework in which Java programs are compiled for a hypothetical instruction set architecture. Programs are distributed in this abstract form, divorced from the details of any other computer architecture. Running a Java program involves either interpreting JVM instructions, compiling them into instructions of the underlying hardware, or directly executing them in a hardware implementation of the JVM.

The JVM is a stack machine. Each instruction gets its operands from the top of a stack, consuming those values and optionally replacing them with a result. The instructions themselves are encoded in a compact

To improve Java program performance, recent implementations use JIT compilation techniques rather than interpretation.

Übersetzung durch simulierte Ausführung

- “As the compiler steps through the bytecodes, it essentially simulates their execution, keeping information about the values on the operand stack at every point. The compiler uses its own stack, which mirrors the runtime status of the operand stack. Instead of holding runtime values, the compile-time stack records information about the intermediate results of the computation. **If the code to produce a value has not yet been emitted, the entry on the compile-time stack records the operations that need to be performed. Otherwise, it specifies the register or memory location that holds the value.**”

Erläuterung des Beispiels im Originalpapier

“As an example, consider the process of generating SPARC instructions for the bytecodes from Figure 1. As the compiler steps through the first three bytecodes, it pushes three entries onto the compile-time stack. Figure 3 shows the contents of the stack at that point, assuming that local variables y and z have been allocated to registers L2 and L3. **When the compiler encounters the multiplication bytecode, it emits code to perform that operation.** The top two entries on the compiletime stack specify the operands. The constant integer 2 can be included as an immediate operand. A register, L0 in this case, must be selected to hold the intermediate result. The compiler then emits the instruction `smul L3, 2 -> L0` results and updates the compile-time stack, as shown in Figure 4. The compiler processes the addition in a similar manner. If local variable x was allocated to a register (for example, L1), the result of the addition can be placed directly in that register. The result in that case is the instruction `add L2, L0 -> L1`. This example demonstrates how a JIT compiler removes interpretation overhead, reducing the entire expression to as few as two SPARC instructions.”

```
0: iload y
1: iconst 2
2: iload z
3: imul
4: iadd
5: istore x
```

TOP



generated code

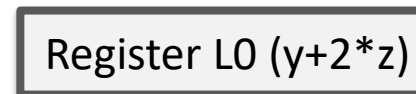
...

TOP



...
`smul L3, 2 -> L0`

TOP



...
`smul L3, 2 -> L0`
`add L2, L0 -> L1`

```
0: iload y
1: iconst 2
2: iload z
3: imul
4: iadd
5: istore x
```


Anmerkungen

- “So far, we have focused on the compilation process within a single basic block; **handling block boundaries** requires some additional effort. Besides holding the intermediate values in an expression evaluation, the operand stack may also hold values across basic block boundaries.”
- “Some bytecodes, such as those involving **memory allocation or synchronization**, are translated into calls to JVM routines.”
- “Method calls in compiled code may invoke the JVM interpreter, either directly by **calling methods that for some reason are not compiled**, or indirectly by calling native methods that in turn invoke the interpreter”
- “To allow compiled and interpreted methods to coexist, we must **translate between the different calling conventions** at every transition between the interpreter and compiled code.”

T. Suganuma *et al.*, in *IBM Systems Journal*, vol. 39, no. 1, pp. 175-193, 2000, doi: 10.1147/sj.391.0175.



Figure 1 JIT compiler overall structure

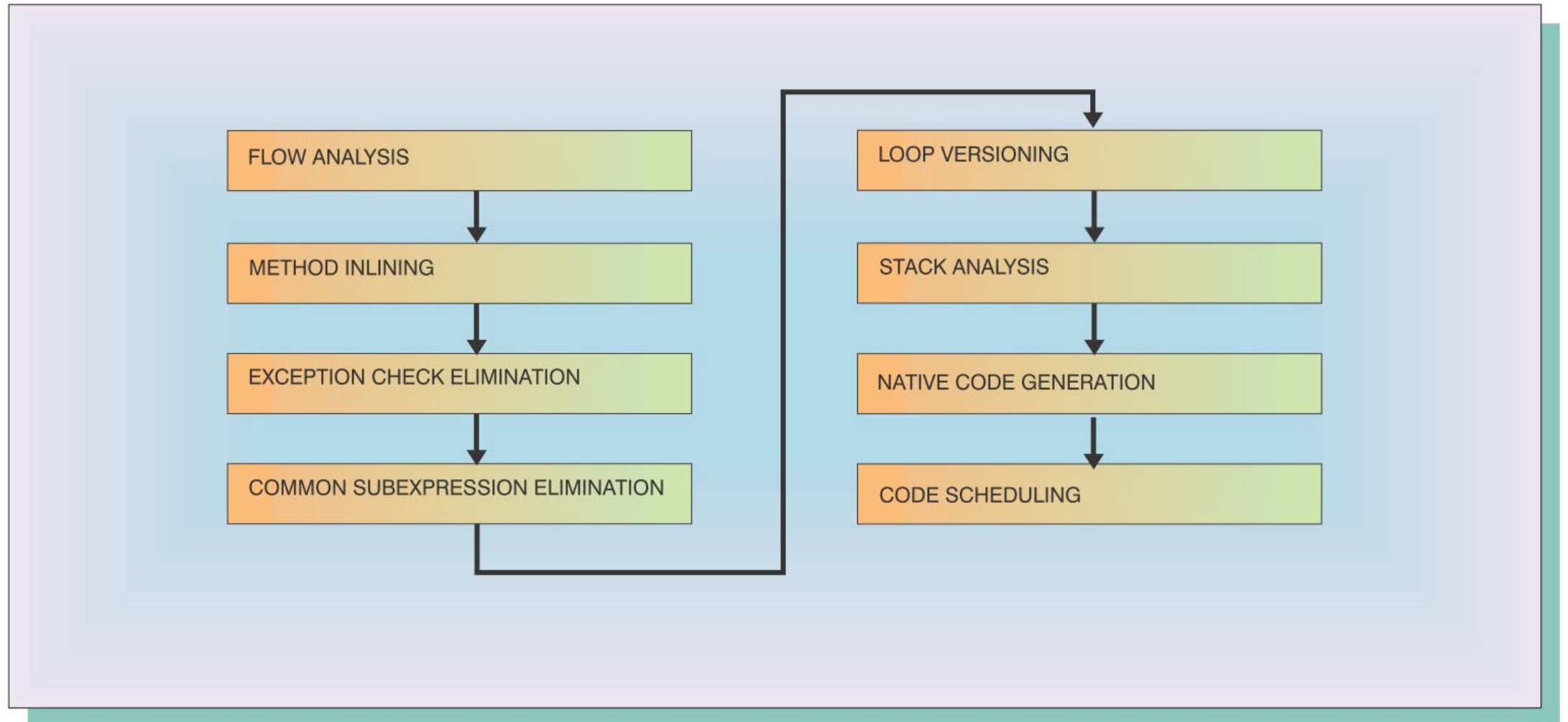


Figure 3 Examples of method inlining

```
public static void foo (x, y, z) {
  if (x==0)
    return (y+z);
  else
    return foo (x-1, y+1, z-1);
}
```



```
public static void foo (x, y, z) {
  BEGIN: if (x==0)
    return (y+z);
  else {
    x' = x-1; y' = y+1; z' = z-1;
    if (x'==0)
      return (y'+z');
    else {
      x = x'-1; y = y'+1; z = z'-1;
      goto BEGIN;
    }
  }
}
```

(A) Tail recursion elimination

```
if(o-> methodtable [mb -> offset] == mb) {
  /* inlined code of mb */
} else {
  /* original virtual invocation code */
  o.mb (a, b, c)
}
```

(B) Virtual method inlining

```

class example {
  int a[ ];
  void sortTest ( ) {
    for (int i = a.length; --i >= 0;) {
      for (int j = 0; j < i; j++) {
        if (a[j] > a[j+1]) {
          int tmp = a[j];
          a[j] = a[j+1];
          a[j+1] = tmp;
        }
      }
    }
  }
}

```

(A) Original program

```

class example {
  int a [ ];
  void sortTest ( ) {
    int la [ ] = a;
    for (int i = la.length; --i(>=0;) {
      for (int j = 0; j<i; j++) {
        int *la0 = &la [j];
        int iv0 = *la0;
        int iv1 = *(la0+1);
        if (iv0>iv1) {
          *la0 = iv1;
          *(la0+1) = iv0;
        }
      }
    }
  }
}

```

(B) Optimized pseudocode (redundancy elimination applied)

```

19: eaload_5 (0) / istore_6
21: eaload_5 (1) / istore_7
23: iload_6 / iload_7 / icmp2 30
BB:
26: iload_6 / eastore_5 (1)
28: iload_7 / eastore_5 (0)
BB:
30: iinc2 1
31: iload_2 / iload_1 / icmp2 15

```

(C) Optimized extended bytecode (combined as idioms)

register allocation

local var 1 → edx (var i)
 local var 2 → edi (var j)
 local var 4 → ebx (var la [])

```

store_5
15: mov eax, [ ebx ]
    lea esi, [ ebx + edi*4 + 8]
    dec eax
    js_throw_out_of_bound exception
    cmp eax, edi
    jbe_throw_out_of_bound exception
19: mov ecx, [esi]
21: mov eax, [esi + 4]
23: cmp eax, ecx
    jge 30
26: mov [esi], eax
28: mov [esi + 4], ecx
30: inc edi
31: cmp edi, edx
    jlt 15

```

(D) Generated native code

Quelle: “Visual Amortization Analysis of Recompilation Strategies”, S. Zimmer and S. Diehl, in *Proceedings of 14th International Conference Information Visualisation*, 2010

2010 14th International Conference Information Visualisation

Visual Amortization Analysis of Recompilation Strategies

Stephan Zimmer and Stephan Diehl (Authors)
Computer Science Department
University of Trier
Trier, Germany
Email: zimmer23@web.de, diehl@uni-trier.de

Abstract—Dynamic recompilation tries to produce more efficient code by exploiting runtime information. Virtual machines like the Jikes RVM use recompilation heuristics to decide how to recompile the program, i.e. what parts are recompiled at what level of optimization. In this paper we present our post-mortem amortization analysis based on improved call stack sampling. Our tool presents the results of the analysis as an interactive visualization to help both virtual machine implementers improve their recompilation strategies, as well as programmers assess whether those recompilation strategies pay off not only for their application as a whole, but also for individual methods.

Keywords—software visualization; recompilation; dynamic analysis

1. INTRODUCTION

The Jikes RVM [1] has been developed as a research platform to experiment with different approaches to implement virtual machines. One of the most important features of the Jikes RVM is its adaptive optimization system [2], [3] which based on measurements of runtime performance, decides when and how to recompile functions. Obviously, recompilation requires extra time at runtime, and thus a method should only be recompiled, if enough time is saved by future executions of the resulting, optimized code. In other words, the optimized code must be faster than the old code, and it must be executed a sufficient number of times later on. Unfortunately, in general the number of future executions of a method is not known at runtime, thus the Jikes RVM uses heuristics to control recompilation of methods.

From a programmer's point of view the question is not, whether these recompilation heuristics of the Jikes RVM pay off for many applications, but whether they really help to speed up his or her application. To this end we developed the post-mortem amortization analysis and visualization methods presented below and integrated these as a plugin into the Eclipse IDE.

Our approach to analyze the amortization of recompilation for a given application program consists of the following four steps:

1. *Instrumenting the byte-code*: The instrumentation computes the dynamic call graph and exact call counts for

each method. In addition, the time spent in instrumentation code¹ is recorded.

2. *Executing the code on an extended Jikes RVM*: The extended virtual machine computes partial context trees.

3. *Analyzing recorded data post-mortem*: Based on the information in the partial context tree and the call counts, estimations of execution times can be computed after the execution of the application.

4. *Visualizing the results*: The overall amortization results are shown in the dynamic call graph, while recompilation diagrams present detailed information for each method.

For estimation of the execution times of each method we use call stack sampling. To this end we extended the Jikes RVM, such that at regular intervals a sampling thread interrupts the execution of the application and inspects its call stack. Our estimation is based on the assumption that methods with short execution times are less likely to be found on the stack than those with longer execution times.

II. RECOMPILATION HEURISTICS

The Jikes RVM uses the following recompilation heuristic: A method is recompiled, if the sum of the expected compilation time and the expected future execution time of the optimized method, is lower than the execution time of the method, so far. More precisely:

$$C_{m,j}^{\text{exp}} + T_{m,j}^{\text{exp}} < T_{m,j}^{\text{act}} \quad \text{where } i, j \in \{0, 1, 2, 3\} \text{ and } i > j \quad (1)$$

Here, $C_{m,j}^{\text{exp}}$ is the expected compilation time for method m at optimization level i . Level 0 corresponds to the Jikes RVM BASE level compilation, 1 to OPT0, 2 to OPT1, and 3 to OPT2. $T_{m,j}^{\text{exp}}$ is the expected execution time of method m at optimization level i , and $T_{m,j}^{\text{act}}$ is the actual execution time of method m at the current optimization level j .

To compare the expected execution time, the Jikes RVM assumes that the method is executed in the optimized code at least as often, as it has been executed, so far.

III. AMORTIZATION OF RECOMPILATION

Once the execution of the application has finished, the recompilation of a method at level i actually paid off, if

¹Here, the instrumentation code itself is excluded from optimization.

Jikes RVM: Recompilation Heuristics

- Heuristik: Eine Methode wird rekompiliert, wenn die Summe aus **erwarteter Übersetzungszeit C** und **erwarteter, zukünftiger Ausführungszeit T** der optimierten Methode geringer ist als die bisherige Ausführungszeit der Methode.

$$C_{m,i}^{exp} + T_{m,i}^{exp} < T_{m,j}^{act} \quad \text{where } i, j \in \{0, 1, 2, 3\} \text{ and } i > j$$

- Der Optimierungsgrad i ist der Jikes Optimization Level:
 - BASE $i=0$, OPT0 $i=1$, OPT1 $i=2$, OPT2 $i=3$
- Die Rekompilierung hat sich amortisiert, wenn $C_{m,i}^{act} + T_{m,i}^{act} < T_{m,j}^{act}$

Amortisierungsanalyse

For our post-mortem amortization analysis we assume *that each invocation of a method takes the same amount of time*, thus we compute the product of the number of invocations and the average execution time of a method as an estimation of $T_{m,i}^{act}$:

$$C_{m,i}^{act} + (N_{m,i}^{act} * t_{m,i}^{post}) < (N_{m,j}^{act} * t_{m,j}^{post}) \quad (3)$$

$N_{m,i}^{act}$ is the number of times the method m at optimization level i is actually called. It is computed by the instrumentation code. $t_{m,i}^{post}$ is our post-mortem estimation of the average time for a single execution of method m at optimization level i .

Dynamischer Aufrufbaum mit nicht kompilierten Methoden (**blau**), erfolgreich optimierten Methoden (**grün**) und nicht erfolgreich optimierten Methoden (**rot**)

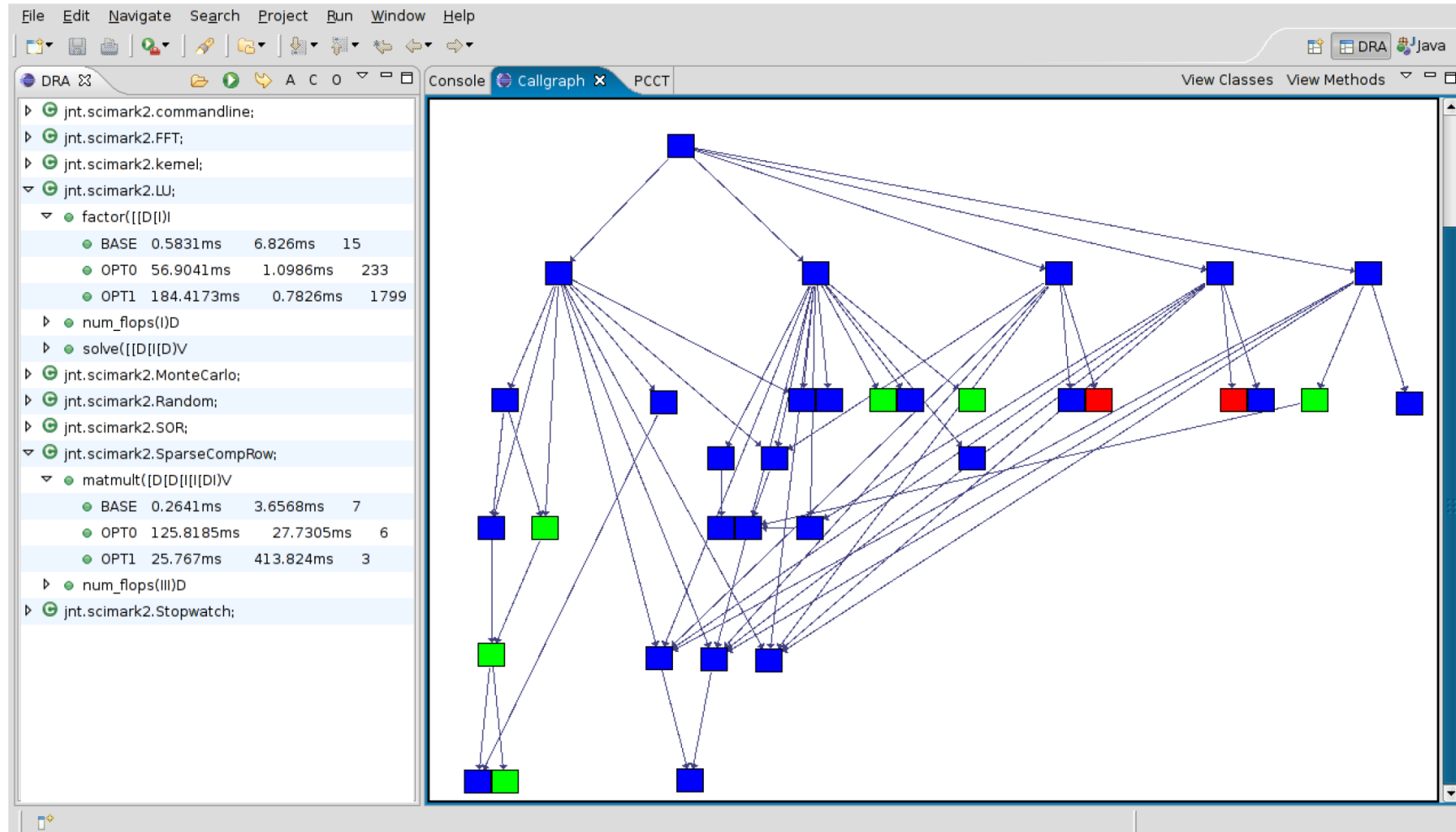


Figure 1. Dynamic call graph

Amortisierungsanalyse

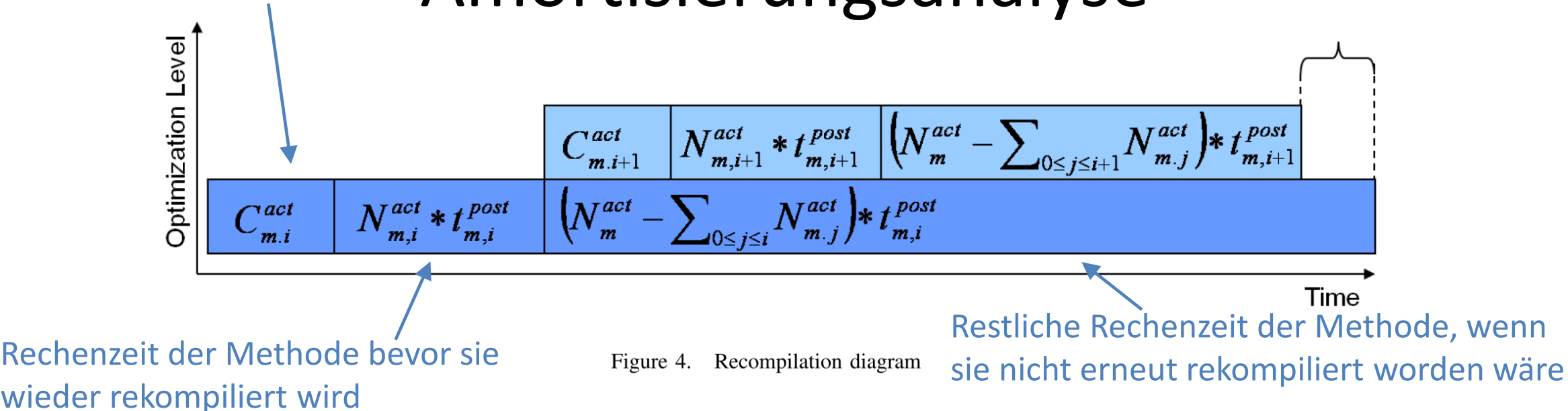
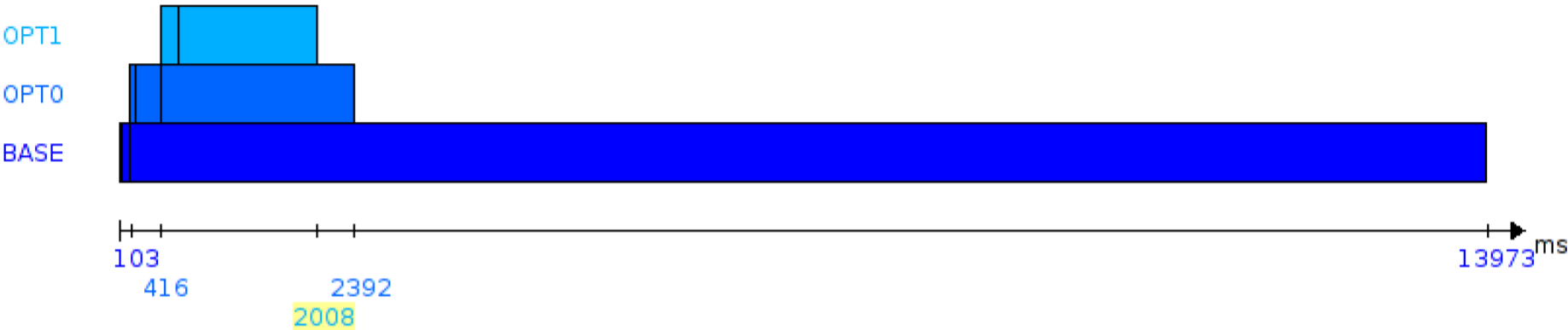


Figure 4. Recompilation diagram

Recompilation Diagram

Class: jnt.scimark2.LU;	[CompileTime	AvgExecTime	CallCount]
Method: factor([D[I])	BASE	0.5831ms	6.826ms 15
	OPT0	56.9041ms	1.0986ms 233
	OPT1	184.4173ms	0.7826ms 1799



Amortisierungsanalyse

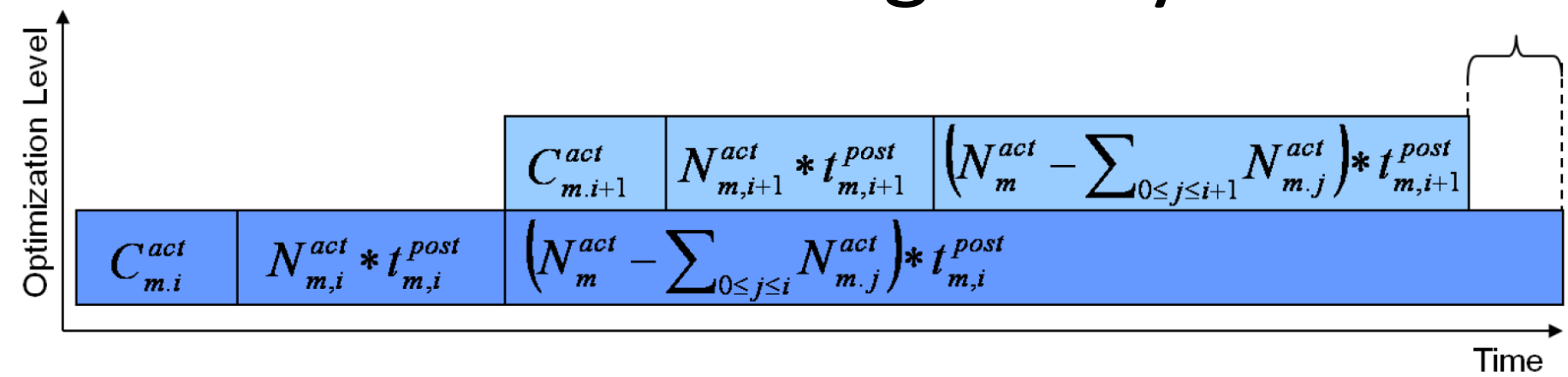


Figure 4. Recompilation diagram

